



Universidade de Brasília
Instituto de Exatas
Departamento de Ciências da Computação
Programa de Pós Graduação em Informática

PCA - Projeto e Complexidade de Algoritmos

Algoritmo de Needleman-Wunsch



Luan Freitas

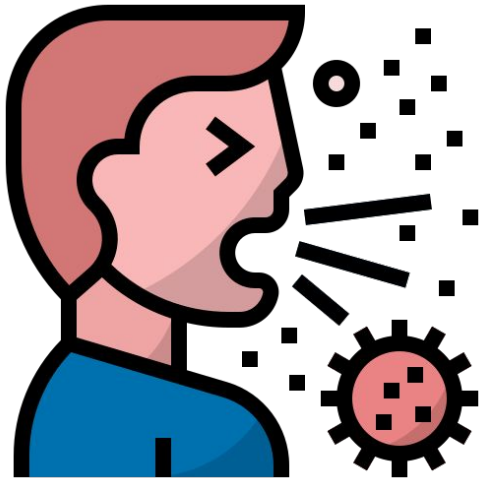


Raul Carvalho

Brasília, 2026

Motivação

Doenças



Organismos Biológicos



Fármacos



Compostos Químicos

Área de Pesquisa

O que é ?

Objetivos

Origem dos dados

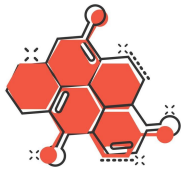
Aplicações



RNA



DNA



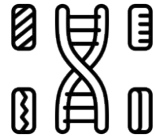
AMINOÁCIDOS



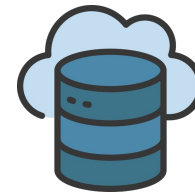
Semelhanças



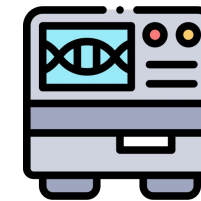
Mutações



Evolução



Banco de Dados



Sequenciadores

Medicina

Genética

Doenças

Bioinformática

Evolução

Engenharia
Genética

Fármacos

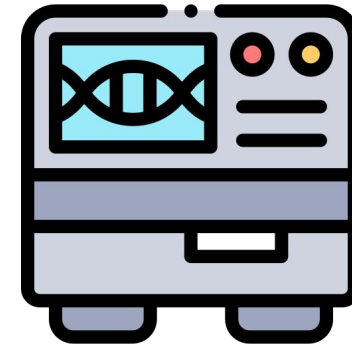
Dados

Banco de Dados



- GenBank (NCBI - EUA)
- European Nucleotide Archive / ENA (EMBL-EBI - Europa)
- DDBJ (DNA Data Bank of Japan - Japão)

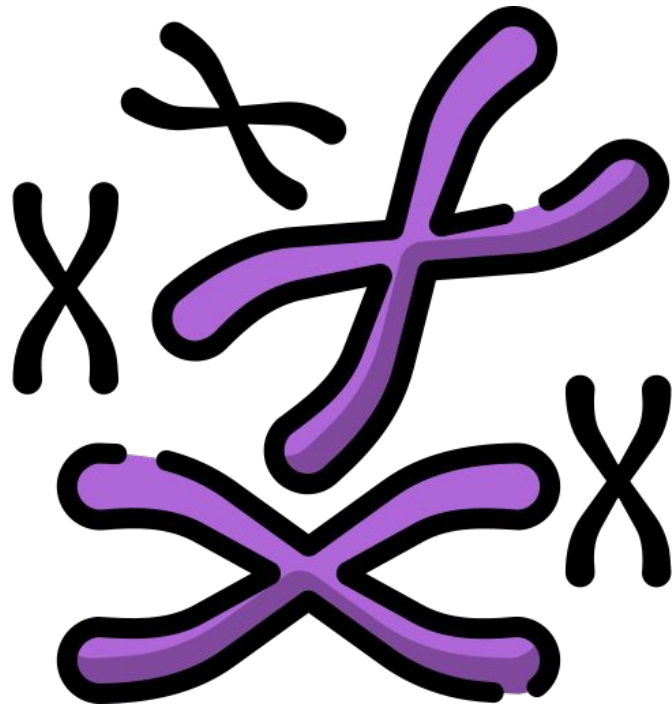
Sequenciadores



- MinION
- GridION
- PromethION
- Vega
- Revio

Solução

Analisar sequências de material genético

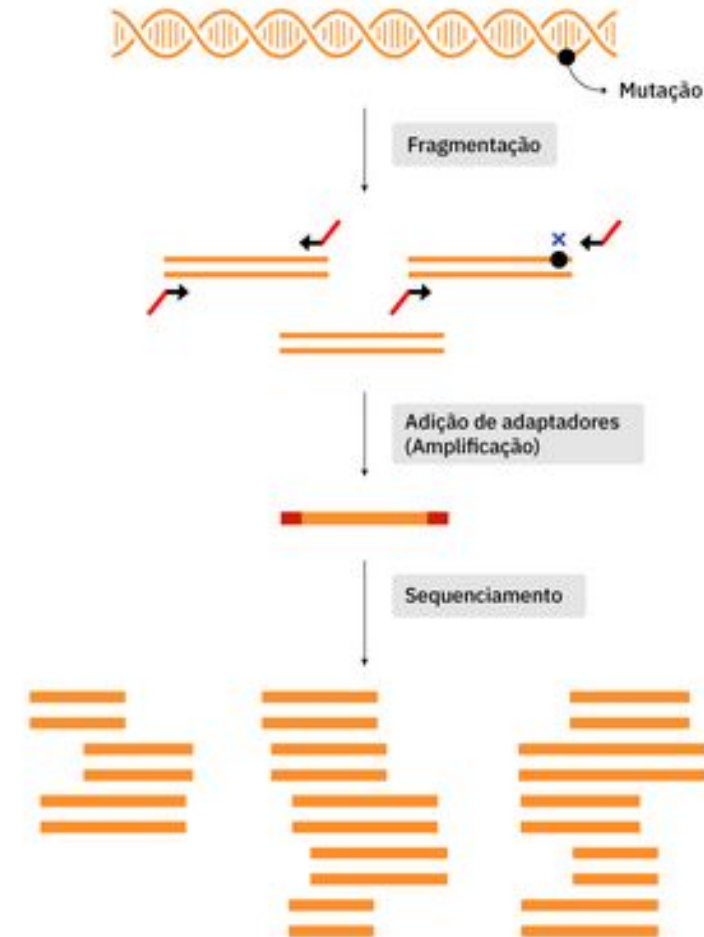


- Formato
- Tamanho
- Organização
- Obtenção
- Montagem
- Análise
- Valor biológico

#PROBLEMAS

Problema Inicial

A comparação de sequências biológicas é um problema fundamental da bioinformática que consiste em avaliar a similaridade entre duas ou mais sequências de DNA, RNA ou proteínas, com o objetivo de identificar relações evolutivas, estruturais ou funcionais entre elas.



Tipos de Sequências

RNA



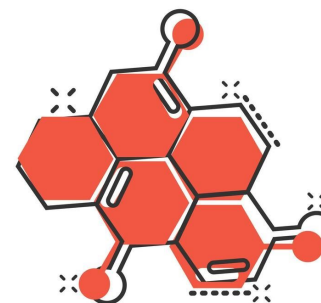
A = Adenina
C = Citosina
G = Guanina
U = Uracila

DNA



A = Adenina
C = Citosina
G = Guanina
T = Timina

AMINOÁCIDOS



A - Alanina
R - Arginina
N - Asparagina
D - Ácido Aspártico
C - Cisteína
E - Ácido Glutâmico
Q - Glutamina
G - Glicina

H - Histidina
I - Isoleucina
L - Leucina
K - Lisina
M - Metionina
F - Fenilalanina
P - Prolina
S - Serina

T - Treonina
W - Triptofano
Y - Tirosina
V - Valina

Trabalhos Base

Artigo Original (1970)

Needleman, S.B. & Wunsch, C.D.

"A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of Two Proteins"

J. Mol. Biol. 48, 443-453 (1970)

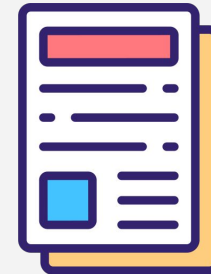


Tese de Doutorado (2015)

Sandes, E.F.O.

"Algoritmos Paralelos Exatos e Otimizações para Alinhamento de Sequências Biológicas Longas em Plataformas de Alto Desempenho"

UnB, 2015



Formulação do Problema

Problema de comparação de sequências biológicas

- **Entrada:** Duas sequências, S1 de tamanho m e S2 de tamanho n .
- **Objetivo:** Encontrar o alinhamento ótimo que maximize semelhanças e minimize diferenças, permitindo identificar regiões conservadas ou funcionais.

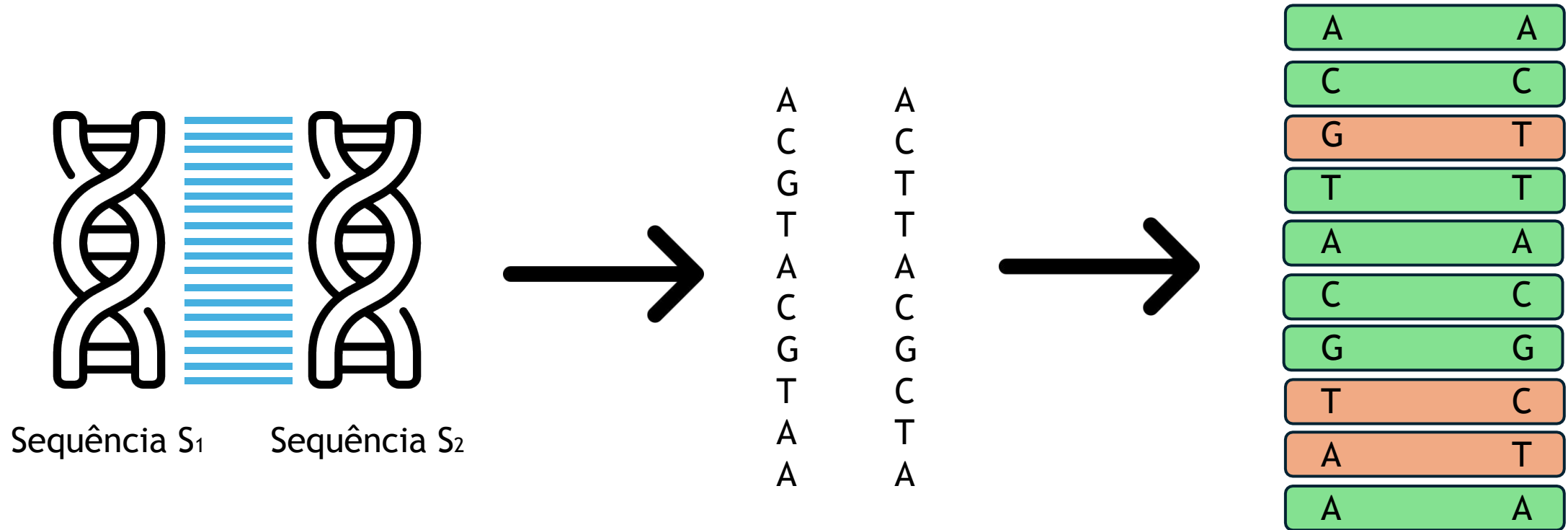
Algoritmo Needleman - Wunsch

O algoritmo de Needleman-Wunsch resolve o alinhamento global de sequências, usando uma função de pontuação com :

- Match
- Mismatch
- Gap

$$F(i, j) = \max \begin{cases} F(i - 1, j - 1) + \text{score}(x_i, y_j) \\ F(i - 1, j) + \text{gap} \\ F(i, j - 1) + \text{gap} \end{cases}$$

Comparação de Sequências



Exemplo

- S1 = AATG
- S2 = ACTG
- Qual é escore ótimo ?

Para cada posição $M[i][j]$, o valor é o **máximo** entre três possibilidades:

1. **Diagonal (match/mismatch)**

$$M[i-1][j-1] + \text{score}(S1[i], S2[j])$$

- Se $S1[i] = S2[j]$: usa **match** (+1).
- Se diferentes: usa **mismatch** (-1).

2. **Cima (gap em S1)**

$$M[i-1][j] + \text{gap}$$

- Gap = -1.

3. **Esquerda (gap em S2)**

$$M[i][j-1] + \text{gap}$$

	SEQ 2	A	C	T	G
SEQ 1	0	-1	-2	-3	-4
A	-1	1	0	-1	-2
A	-2	0	0	-1	-2
T	-3	-1	-1	1	0
G	-4	-2	-2	0	2

Pontuação:

- Matches: $3 \times (+1) = +3$
- Mismatches: $1 \times (-1) = -1$
- Gaps: 0
- **Total = 2**

Abordagem 1 - Puramente recursivo

Implementação: A função chama a si mesma para cada possibilidade (diagonal, cima, esquerda).

Complexidade:

- Tempo: $O(3^m + n)$ (explora todas as combinações possíveis).
- Espaço: apenas a pilha de chamadas, mas pode ser muito profundo.

Problema: Explosão combinatória → inviável para sequências maiores que ~20 caracteres.

Uso típico: Apenas para fins didáticos, para entender a lógica.

```
21 // Algoritmo Recursivo Puro de Needleman-Wunsch.
22 // Calcula o score de alinhamento para prefixos de seqA e seqB de tamanho i e j.
23 // A matriz M_visualizacao é preenchida com valores de pontuação para visualização.
24 int nw_recursivo_puro(const char *seqA, const char *seqB, int i, int j) {
25     // Caso base: seqA esvaziou, preenche coluna j com penalidades de gaps.
26     if (i == 0) {
27         M_visualizacao[0][j] = j * GAP;
28         return j * GAP;
29     }
30     // Caso base: seqB esvaziou, preenche linha i com penalidades de gaps.
31     if (j == 0) {
32         M_visualizacao[i][0] = i * GAP;
33         return i * GAP;
34     }
35
36     // Determina se os caracteres atuais formam match ou mismatch.
37     int score_match = (seqA[i - 1] == seqB[j - 1]) ? MATCH : MISMATCH;
38
39     // Explora as três possibilidades de transição recursiva:
40     // 1) diagonal (alinhamento dos dois caracteres)
41     // 2) deleção (gap em seqB)
42     // 3) inserção (gap em seqA)
43     int diagonal = nw_recursivo_puro(seqA, seqB, i - 1, j - 1) + score_match;
44     int delecao = nw_recursivo_puro(seqA, seqB, i - 1, j) + GAP;
45     int insercao = nw_recursivo_puro(seqA, seqB, i, j - 1) + GAP;
46
47     int resultado = max3(diagonal, delecao, insercao);
48     M_visualizacao[i][j] = resultado;
49
50     return resultado;
51 }
```

$$T(m,n)=T(m-1,n-1)+T(m-1,n)+T(m,n-1)+(C_d+C_c+C_e+C_{max}+C_{rec})$$

Abordagem 2 - Recursivo Memoizado

Implementação: Guardar resultados já calculados em uma tabela (cache).

Complexidade:

- Tempo: $O(m \cdot n)$, pois cada subproblema é resolvido uma vez.
- Espaço: $O(m \cdot n)$ para armazenar a tabela.

Vantagem: Muito mais rápido que o recursivo puro.

Problema: Ainda depende da pilha de recursão, podendo estourar em sequências muito grandes.

```
21
22 // Algoritmo de Needleman-Wunsch recursivo com memoização (top-down  $O(m \times n)$ ).
23 // Calcula o score de alinhamento para prefixos de seqA e seqB de comprimento i e j.
24 int nw_recursivo_memoizado(const char *seqA, const char *seqB, int i, int j) {
25     // Caso base: seqA está vazia, preencher a primeira linha com gaps.
26     if (i == 0) {
27         M_visualizacao[0][j] = j * GAP;
28         return j * GAP;
29     }
30     // Caso base: seqB está vazia, preencher a primeira coluna com gaps.
31     if (j == 0) {
32         M_visualizacao[i][0] = i * GAP;
33         return i * GAP;
34     }
35
36     // Se o valor já foi calculado antes, retorna diretamente para evitar recomputação.
37     if (M_visualizacao[i][j] != UNKNOWN) {
38         return M_visualizacao[i][j];
39     }
40
41     int score_match = (seqA[i - 1] == seqB[j - 1]) ? MATCH : MISMATCH;
42
43     // Calcula recursivamente os três possíveis caminhos de alinhamento.
44     int diagonal = nw_recursivo_memoizado(seqA, seqB, i - 1, j - 1) + score_match;
45     int delecao = nw_recursivo_memoizado(seqA, seqB, i - 1, j) + GAP;
46     int insercao = nw_recursivo_memoizado(seqA, seqB, i, j - 1) + GAP;
47
48     int resultado = max3(diagonal, delecao, insercao);
49     M_visualizacao[i][j] = resultado;
50
51     return resultado;
52 }
```

$$T(m,n)=m \cdot n \cdot (C_d+C_c+C_e+C_{\max}+C_{\text{mem}})$$

Abordagem 3 - Programação Dinâmica

Implementação: Preencher iterativamente uma matriz de tamanho $m \times n$.

Complexidade:

- Tempo: $O(m \cdot n)$.
- Espaço: $O(m \cdot n)$, mas pode ser otimizado para $O(\min(m, n))$ se só o score for necessário.

Vantagem: Mais eficiente e estável, sem risco de estouro de pilha.

Uso típico: Implementação prática em bioinformática.

```
16 // Implementação de Needleman-Wunsch por programação dinâmica.
17 // Preenche a matriz de pontuações H e reconstrói o caminho ótimo.
18 void nw_programacao_dinamica(const char *seqA, const char *seqB) {
19     int m = strlen(seqA);
20     int n = strlen(seqB);
21
22     // Aloca as matrizes de pontuação e de traceback com dimensões (m+1) x (n+1).
23     int **H = (int **)malloc((m + 1) * sizeof(int *));
24     int **Caminho_Traceback = (int **)malloc((m + 1) * sizeof(int *));
25     for (int i = 0; i <= m; i++) {
26         H[i] = (int *)calloc((n + 1), sizeof(int));
27         Caminho_Traceback[i] = (int *)calloc((n + 1), sizeof(int));
28     }
29
30     // Inicializa as bordas da matriz com penalidades de gap.
31     for (int i = 0; i <= m; i++) H[i][0] = i * GAP;
32     for (int j = 0; j <= n; j++) H[0][j] = j * GAP;
33
34     // Calcula os valores da matriz linha a linha, usando os subproblemas resolvidos previamente.
35     for (int i = 1; i <= m; i++) {
36         for (int j = 1; j <= n; j++) {
37             int score_match = (seqA[i - 1] == seqB[j - 1]) ? MATCH : MISMATCH;
38
39             H[i][j] = max3(
40                 H[i - 1][j - 1] + score_match,
41                 H[i - 1][j] + GAP,
42                 H[i][j - 1] + GAP
43             );
44         }
45     }
46 }
```

$$T(m,n)=m \cdot n \cdot (C_d+C_c+C_e+C_{\max}+C_{dp})$$

Cálculo de Complexidade

- m: tamanho da sequência S1
- n: tamanho da sequência S2
- Cd: custo de calcular a opção **diagonal** (match/mismatch)
- Cc: custo de calcular a opção **cima** (gap em S1)
- Ce: custo de calcular a opção **esquerda** (gap em S2)
- Cmax: custo de escolher o máximo entre as três opções
- Crec: custo de chamada recursiva
- Cmem: custo de acesso ao cache/memoização
- Cdp: custo de preencher uma célula na matriz dinâmica

Método	Fórmula genérica	Melhor caso	Caso médio	Pior caso
Recursivo puro	$T(m,n)=T(m-1,n-1)+T(m-1,n)+T(m,n-1)+(Cd+Cc+Ce+Cmax+Crec)$	Exponencial $O(3^{\min(m,n)})$	Exponencial $O(3^{(m+n)})$	Exponencial $O(3^{(m+n)})$
Recursivo com memoização	$T(m,n)=m \cdot n \cdot (Cd+Cc+Ce+Cmax+Cmem)$	$O(m \cdot n)$	$O(m \cdot n)$	$O(m \cdot n)$
Programação dinâmica	$T(m,n)=m \cdot n \cdot (Cd+Cc+Ce+Cmax+Cdp)$	$O(m \cdot n)$	$O(m \cdot n)$	$O(m \cdot n)$

Comparação de Complexidade

Método	Tempo de execução	Espaço de memória	Características	Usabilidade
Recursivo puro	Exponencial $O(3^{(m+n)})$	Baixo (apenas pilha de chamadas)	Explora todas as possibilidades sem reaproveitar cálculos	Didático, mas impraticável para sequências grandes
Recursivo com memoização	Polinomial $O(m \cdot n)$	$O(m \cdot n)$ para cache	Reaproveita resultados já calculados, evita recomputações	Bom para sequências médias, mas limitado pela pilha
Programação dinâmica	Polinomial $O(m \cdot n)$	$O(m \cdot n)$, podendo otimizar para $O(\min(m,n))$	Preenche iterativamente uma matriz, sem recursão	Padrão inicial em bioinformática, eficiente e estável

Comparação de Complexidade

Método	Melhor caso	Caso médio	Pior caso	Observações
Recursivo puro	$O(3^{(m+n)})$	$O(3^{(m+n)})$	$O(3^{(m+n)})$	Cada célula pode gerar até 3 chamadas recursivas; mesmo em melhor caso não há otimização
Recursivo com memoização	$O(m \cdot n)$ operações (cada subproblema resolvido uma vez)	$O(m \cdot n)$	$O(m \cdot n)$	Cache evita recomputações; custo é linear no número de células da matriz. total $\approx 4 \cdot m \cdot n$ operações
Programação dinâmica	$O(m \cdot n)$	$O(m \cdot n)$	$O(m \cdot n)$	Cada célula exige 3 comparações + 1 máximo; total $\approx 4 \cdot m \cdot n$ operações

Caso Prático

Método	m = n = 10	m = n = 100	m = n = 1000	Observações
Recursivo puro	$\approx 3^{20} \approx 3,486,784,401$ operações	$\approx 3^{200}$ (inviável, número astronômico)	$\approx 3^{2000}$ (completamente impraticável)	Crescimento exponencial, inviável para qualquer sequência real
Recursivo com memoização	$\approx 10 \cdot 10 = 100$ células $\rightarrow$ ~ 400 operações	$\approx 100 \cdot 100 = 10,000$ células $\rightarrow$ $\sim 40,000$ operações	$\approx 1000 \cdot 1000 = 1,000,000$ células $\rightarrow$ $\sim 4,000,000$ operações	Cada célula calculada uma vez, custo linear no tamanho da matriz
Programação dinâmica	≈ 100 células $\rightarrow$ ~ 400 operações	$\approx 10,000$ células $\rightarrow$ $\sim 40,000$ operações	$\approx 1,000,000$ células $\rightarrow$ $\sim 4,000,000$ operações	Mesmo custo que memoização, mas sem overhead de recursão

Hoje

O problema não parou por aqui, porque com o avanço das máquinas de sequenciamento genético, hoje temos acesso a sequências com milhões de bases para serem comparadas, o que torna a aplicação do algoritmo de Needleman-Wunsch inviável por tamanho da matriz e tempo de processamento.

Cromossomo 11 humano (*Homo sapiens*) possui aproximadamente 135 milhões de pares de bases



O genoma da Borboleta azul-do-atlas (*Polyommatus atlantica*), nativa do Norte da África, possui 695 milhões de pares de bases



O cromossomo A1 é o maior cromossomo dos gatos domésticos (*Felis catus*). Ele é um cromossomo de grande porte e seu tamanho é 239 milhões de pares de bases.



Obrigado

"O objetivo principal da educação é criar pessoas capazes de fazer coisas novas e não simplesmente repetir o que as outras gerações fizeram." — Jean Piaget



<https://github.com/luandkg/PCA.git>